

Support de cours synthétique d'apprentissage des bases du langage JAVA

Niveau : B2

Etablissement : KEYCE ACADEMY

PARTIE 1. Récapitulatif des bases du langage.

I- Introduction

Java est un langage de programmation orienté objet, polyvalent et basé sur la plateforme, conçu pour être simple, sûr et portable. Il est largement utilisé pour le développement d'applications web, mobiles et d'entreprise.

a- Caractéristiques Principales

- **Portabilité** : "Écrire une fois, exécuter partout" grâce à la machine virtuelle Java (JVM).
- **Sécurité** : Exécution dans un environnement sécurisé.
- **Multithreading** : Support natif pour l'exécution simultanée de plusieurs threads.

II- Syntaxe de Base

Un programme Java est constitué de classes, mais même sans POO, nous pouvons écrire un programme simple.

```
public class MonProgramme {  
    public static void main(String[] args) {  
        System.out.println("Bonjour, monde !");  
    }  
}
```

III- Variables

Les variables sont des conteneurs pour stocker des valeurs.

Syntaxe générale de déclaration des variables

TypeDeDonnée variable = valeur ; // déclaration avec initialisation.

Exemple :

```
int age = 25;  
double prix = 19.99;  
char initial = 'J';  
boolean estActif = true;
```

IV- Types de Données

Java est un langage typé. Les types de données sont classés en deux catégories : primitifs et objets.

A- Types Primitifs

int	Manipule les entiers (positifs ou négatifs)
double	Manipule les valeurs réelles de grandes tailles (positifs ou négatives y compris les valeurs entières)
char	Manipule les caractères.
boolean	Stocke les valeurs booléennes : true ou false.
float	Manipule les valeurs réelles (positifs ou négatives y compris les valeurs entières)
long	Manipule les valeurs entières de grandes tailles (positifs ou négatives y compris les valeurs entières)
Etc..	

V- Les opérateurs

A- Opérateurs Arithmétiques

Addition : +	
Soustraction : -	
Multiplication : *	
Division : /	
Modulo : %	

B- Opérateurs logiques

Les opérateurs logiques en Java sont utilisés pour combiner des expressions booléennes et pour effectuer des opérations logiques. Ils sont essentiels pour le contrôle de flux dans les structures conditionnelles, comme les instructions if, while, et les boucles. Voici un aperçu des principaux opérateurs logiques en Java.

ET : &&	Description Renvoie <code>true</code> si les deux opérandes sont vrais. Sinon, il renvoie <code>false</code>. Souvent utilisé pour vérifier plusieurs conditions simultanément. boolean variable=condition1 && condition2 ; Exemple <pre> public class Utilitaires { public static void main(String[] args) { boolean condition1 = true; boolean condition2 = false; if (condition1 && condition2) { System.out.println("Les deux conditions sont vraies."); } else { System.out.println("Au moins une condition est fausse."); // Affiché } } } </pre>
OU : 	Description Renvoie <code>true</code> si au moins un des opérandes est vrai. Renvoie <code>false</code> seulement si les deux opérandes sont faux. Utilisation Utilisé pour vérifier si au moins une condition est vraie. Syntaxe boolean variable=condition1 condition2 ; // condition1 et condition2 sont des variables booléennes Exemple <pre> public class Utilitaires { public static void main(String[] args) { boolean condition1 = true; boolean condition2 = false; if (condition1 condition2) { System.out.println("Au moins une condition est vraie."); // Affiché } else { System.out.println("Les deux conditions sont fausses."); } } } </pre>
NON : !	Description Inverse la valeur de vérité de l'opérande. Si l'opérande est vrai, il renvoie <code>false</code>, et vice versa. Utilisation Utilisé pour vérifier l'opposée d'une condition. Syntaxe boolean variable1= !condition1 ; // condition1 est une variable de type boolean Exemple

	<pre> 1 public class Utilitaires { 2 3 public static void main(String[] args) { 4 boolean condition = false; 5 6 if (!condition) { 7 System.out.println("La condition est fausse."); // Affiché 8 } 9 } 10 }</pre>
Priorité des Opérateurs Logiques	Description Les opérateurs logiques ont une certaine priorité qui détermine l'ordre d'évaluation. Les opérateurs ! ont la plus haute priorité, suivis de &&, puis de . Cela signifie que les expressions contenant plusieurs opérateurs logiques seront évaluées selon cette priorité. Exemple de Priorité <pre> public class Utilitaires { public static void main(String[] args) { boolean a = true; boolean b = false; boolean c = true; if (a b && c) { System.out.println("Condition vraie."); // Affiché } } }</pre> Dans cet exemple, b && c est évalué d'abord, puis le résultat est combiné avec a.

C- Opérateurs de Comparaison

Les opérateurs de comparaison en Java sont utilisés pour comparer deux valeurs. Ils renvoient un résultat booléen (true ou false) en fonction de la relation entre les valeurs comparées. Voici un aperçu des principaux opérateurs de comparaison en Java.

Égal à : ==	Description Vérifie si deux valeurs sont égales. Utilité Utilisé pour comparer des types primitifs ou des références d'objets Exemple
--------------------	---

	<pre> public class Utilitaires { public static void main(String[] args) { int a = 5; int b = 5; if (a == b) { System.out.println("a est égal à b."); // Affiché } } } </pre>
Différent de : !=	Description Vérifie si deux valeurs ne sont pas égales. Utilisation Utilisé pour déterminer si deux valeurs sont différentes. Exemple <pre> public class Utilitaires { public static void main(String[] args) { int a = 5; int b = 6; if (a != b) { System.out.println("a est différent de b."); // Affiché } } } </pre>
Supérieur à : >	Description Vérifie si la première valeur est supérieure à la seconde. Utilisation Utilisé pour comparer des valeurs numériques. Exemple <pre> int a = 7; int b = 5; if (a > b) { System.out.println("a est supérieur à b."); // Affiché } </pre>
Inférieur à : <	Description Vérifie si la première valeur est inférieure à la seconde. Utilisation Utilisé pour comparer des valeurs numériques.
Supérieur ou égal à : >=	Description Vérifie si la première valeur est supérieure ou égale à la seconde.
Inférieur ou égal à : <=	Description Vérifie si la première valeur est inférieure ou égale à la seconde.

VI- Les structures conditionnelles

Encadrant : BATCHATO MEDJONY FAGUY

Ingénieur informaticien & formateur en développement et en bonnes pratiques de développement

Instruction if	- Description L'instruction if est utilisée pour exécuter un bloc de code si une condition spécifiée est vraie. - Syntaxe if (condition) { // Bloc de code à exécuter si la condition est vraie } - Exemple int age = 20; if (age >= 18) { System.out.println("Vous êtes adulte."); }
Instruction if-else	- Description L'instruction if-else permet d'exécuter un bloc de code si la condition est vraie, et un autre bloc si elle est fausse. - Syntaxe if (condition) { // Bloc de code si la condition est vraie } else { // Bloc de code si la condition est fausse } - Exemple <pre>public static void main(String[] args) { System.out.println("Bonjour, monde !"); int age = 16; if (age >= 18) { System.out.println("Vous êtes adulte."); } else { System.out.println("Vous êtes mineur."); } }</pre>
if-else else - if	- Description L'instruction if-else permet d'exécuter un bloc de code si la condition est vraie, et un autre bloc si elle est fausse. - Syntaxe if (condition1) { // Bloc de code si condition1 est vraie } else if (condition2) { // Bloc de code si condition2 est vraie } else { // Bloc de code si aucune condition n'est vraie } - Exemple <pre>1 public class MonProgramme { 2 public static void main(String[] args) { 3 int note = 85; 4 if (note >= 90) { 5 System.out.println("A"); 6 } else if (note >= 80) { 7 System.out.println("B"); 8 } else if (note >= 70) { 9 System.out.println("C"); 10 } else { 11 System.out.println("D"); 12 } 13 } 14 }</pre>
Instruction switch	- Description

	- L'instruction switch permet de tester une expression contre plusieurs valeurs possibles. C'est souvent plus lisible que plusieurs instructions if-else imbriquées. - Syntaxe <pre>switch (expression) { case valeur1: // Bloc de code si expression == valeur1 break; case valeur2: // Bloc de code si expression == valeur2 break; default: // Bloc de code si aucune des valeurs ne correspond }</pre> - Exemple <pre>1 public class MonProgramme { 2 public static void main(String[] args) { 3 int jour = 3; 4 switch (jour) { 5 case 1: 6 System.out.println("Lundi"); 7 break; 8 case 2: 9 System.out.println("Mardi"); 10 break; 11 case 3: 12 System.out.println("Mercredi"); 13 break; 14 default: 15 System.out.println("Autre jour"); 16 } 17 } 18 } 19 }</pre>
--	--

VII- Les boucles

Boucle for	- Description La boucle for est utilisée lorsque le nombre d'itérations est connu à l'avance. Elle est souvent utilisée pour parcourir des tableaux ou des séquences. - Syntaxe <pre>for (initialisation; condition; mise à jour) { // Bloc de code à exécuter à chaque itération }</pre> - Exemple <pre>public class MonProgramme { public static void main(String[] args) { for (int i = 0; i < 5; i++) { System.out.println("Itération " + i); } } }</pre>
-------------------	--

Boucle while	- Description La boucle while est utilisée lorsque le nombre d'itérations n'est pas connu à l'avance. Elle continue d'exécuter le bloc de code tant qu'une condition est vraie. - Syntaxe while (condition) { // Bloc de code à exécuter tant que la condition est vraie } - Exemple <pre> public class MonProgramme { public static void main(String[] args) { int compteur = 0; while (compteur < 5) { System.out.println("Compteur " + compteur); compteur++; } } } </pre>
Boucle do-while	- Description La boucle do-while est similaire à la boucle while, mais elle garantit que le bloc de code s'exécute au moins une fois, car la condition est vérifiée après l'exécution du bloc. - Syntaxe do { // Bloc de code à exécuter } while (condition); - Exemple <pre> public class MonProgramme { public static void main(String[] args) { int j = 0; do { System.out.println("Valeur de j : " + j); j++; } while (j < 5); } } </pre>

VIII- Utilisation de break et continue

a- Break

L'instruction break est utilisée pour sortir immédiatement d'une boucle, quel que soit l'état de la condition.

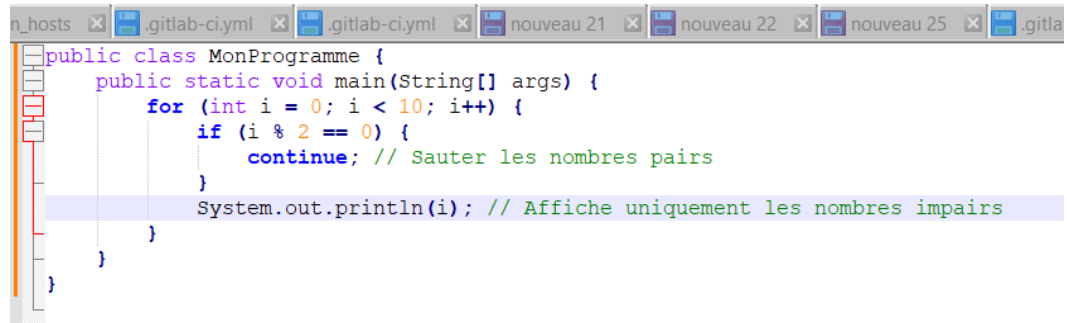
```

1 public class MonProgramme {
2     public static void main(String[] args) {
3         for (int i = 0; i < 10; i++) {
4             if (i == 5) {
5                 break; // Sortie de la boucle lorsque i est 5
6             }
7             System.out.println(i);
8         }
9     }
10 }
11

```

b- Continue

L'instruction continue est utilisée pour sauter l'itération courante d'une boucle et passer à la suivante.



```

public class MonProgramme {
    public static void main(String[] args) {
        for (int i = 0; i < 10; i++) {
            if (i % 2 == 0) {
                continue; // Sauter les nombres pairs
            }
            System.out.println(i); // Affiche uniquement les nombres impairs
        }
    }
}

```

IX- Les tableaux

Description	Les tableaux sont des structures de données qui stockent plusieurs valeurs du même type.
Déclaration et Initialisation	<pre>typededonnee[] nom_tableau = {valeur1, valeur2, valeur3, valeur4, valeur5};</pre> <pre>typededonnees[] nom_tableau2=new typededonnees[nombre_elements_tableau] ;</pre> - Exemple : <pre>double[] montants= {15.0, 21.5, 20.5, 20.5};</pre> <pre>String[] tableauChaines={"FAGUY", "PAMELA", "DUVAL", "HONORINE"};</pre> <pre>double[] tableau=new double[8];</pre> <pre>int[] tableau2=new int[8];</pre> <pre>String[] tableauChaines3=new String[10];</pre>
Accès aux Éléments	Les éléments d'un tableau sont accessibles via leur index (commençant à 0). Syntaxe générale <pre>Variable_tableau[index];</pre> Exemple <pre>tableauChaines[2]="ENEO";</pre> <pre>String nom=tableauChaines[0];</pre>
Longueur du Tableau	<pre>Int longueur=Variable_tableau.length ;</pre>
Boucle sur un Tableau	Utilisez une boucle pour parcourir un tableau. <pre>String[] tableauChaines={"FAGUY", "PAMELA", "DUVAL", "HONORINE"}; tableauChaines[2]="ENEO"; for (int i = 0; i < tableauChaines.length; i++) { System.out.println(tableauChaines[i]); }</pre>

X- Les chaines de caractères (String)

XI- Méthodes

Description	Une méthode est un bloc de code qui effectue une tâche spécifique. Les méthodes permettent de structurer le code, de le rendre plus lisible et de favoriser la réutilisation du code. Elles peuvent prendre des paramètres en entrée et renvoyer une valeur.
Déclaration d'une Méthode	La déclaration d'une méthode se fait directement dans le bloc d'une classe(Comme la méthode principale main vue dans le cours). Avec paramètre <pre> modifier_de_visibilité type_de_retour nom_de_méthode(type param1, type2 param2, type3 param3, etc...) // Bloc de code </pre> - modifier_de_visibilité : Définit la visibilité de la méthode (par exemple, public, private, protected). - type_de_retour : Indique le type de valeur renvoyée par la méthode (par exemple, int, void pour aucune valeur). - nom_de_méthode : Nom que vous donnez à la méthode. - Param1, param2, etc... : Valeurs d'entrée pour la méthode, définies avec un type et un nom. Sans paramètre <pre> modifier_de_visibilité type_de_retour nom_de_méthode() { // Bloc de code } </pre>
Exemple de déclarations de méthodes	Exemple de déclaration de la méthode avec retour. <pre> public static int addition(int a, int b) { return a + b; } </pre> La méthode addition est une méthode qui prend deux paramètres entier a et b et retourne un nombre de type int qui est la somme des deux nombres a+b. Exemple de déclaration du méthode sans valeur de retour. <pre> public static void afficherNombre(int nombre) { System.out.println("Le nombre est : " + nombre); } </pre> afficherNombre() est une méthode sans valeur de retour qui prend en paramètre un nombre de type int et affiche une chaîne de caractère à l'écran ou dans la console. Méthodes sans Paramètres

Il est également possible de créer des méthodes qui ne prennent pas de paramètres.

```
public static void afficherBienvenue() {
    System.out.println("Bienvenue dans le programme !");
}
```

NB : le mot clé **static** dans la définition de la méthode signifie que c'est une méthode de la classe. (Nous verrons en POO. Mais utiliser ce mot clé pour permettre d'appeler la méthode dans la méthode principale main du programme)

```
public class Utilitaires {
    // Méthode pour additionner deux nombres
    public static int addition(int a, int b) {
        return a + b;
    }
    // Méthode pour afficher un message
    public static void afficherMessage(String message) {
        System.out.println(message);
    }
    // Méthode pour calculer la factorielle d'un nombre
    public static int factorielle(int n) {
        if (n < 0) {
            System.out.println("Le nombre doit être positif");
        }
        int resultat = 1;
        for (int i = 1; i <= n; i++) {
            resultat *= i;
        }
        return resultat;
    }
    // Méthode main pour tester les autres méthodes
    public static void main(String[] args) {
        // Tester la méthode addition
        int somme = addition(5, 3);
        afficherMessage("La somme est : " + somme);

        // Tester la méthode factorielle
        int fact = factorielle(5);
        afficherMessage("La factorielle de 5 est : " + fact);
    }
}
```

Appel d'une Méthode

Pour utiliser une méthode, vous devez l'appeler en spécifiant son nom et en fournissant les arguments requis.

Syntaxe 1

nom_de_la_méthode(arg1, arg2, etc....); // appel de methode avec passage de valeur arg1, arg2, etc...

Exemple

int somme=addtion(12, 14) ; // appel de la méthode addition avec passage de deux paramètres entiers 12 et 14. Le valeur de retour de la méthode est affectée à la variable somme.

Syntaxe 2

nom_de_la_méthode(); // appel de methode sans passage de valeur

Exemple

afficherBienvenue() ; // appel de la méthode afficherBienvenue() qui ne se contente que d'afficher à l'écran *bienvenue dans le programme*

	<pre> public class Exemple { public static void main(String[] args) { // Appel d'une méthode avec des arguments int somme = addition(5, 3); // 5 et 3 sont des arguments System.out.println("La somme est : " + somme); // Appel d'une méthode sans arguments afficherMessage(); // Appel d'une méthode sans arguments } // Méthode qui prend deux paramètres et retourne leur somme public static int addition(int a, int b) { return a + b; } // Méthode sans paramètres public static void afficherMessage() { System.out.println("Bonjour, monde !"); } } </pre>
Surcharge de Méthodes	Java permet la surcharge de méthodes, ce qui signifie que vous pouvez avoir plusieurs méthodes avec le même nom mais des paramètres différents. <pre> public static float addition(float a, float b) { return a + b; } public static double addition(double a, double b) { return a + b; } </pre>
Boucle sur un Tableau	Utilisez une boucle pour parcourir un tableau. <pre> String[] tableauChaines={"FAGUY", "PAMELA", "DUVAL", "HONORINE"}; tableauChaines[2]="ENEO"; for (int i = 0; i < tableauChaines.length; i++) { System.out.println(tableauChaines[i]); } </pre>

PARTIE 2. Exercices & TP

I- Exercices pour débutants (**Précision : Chaque étudiant devra faire au moins 10 exercices.**).

Barème :

<10 = -1pt, + de 10exercices = 0.5pt, tous les exercices = 2pts

Exercice 1

Écris un programme qui affiche "Hello, World!" sur la console.

Exercice 2

Demande à l'utilisateur de saisir un nombre entier et vérifie s'il est pair ou impair. Affiche un message approprié.

Exercice 3

Écris un programme qui demande à l'utilisateur de saisir un nombre entier et affiche sa table de multiplication (de 1 à 10).

Exercice 4

Demande à l'utilisateur de saisir un nombre entier positif et calcule sa factorielle. Affiche le résultat.

Exercice 5

Demande à l'utilisateur de saisir un nombre entier n, puis calcule et affiche la somme des nombres de 1 à n.

Exercice 6

Écris un programme qui convertit une température de Celsius à Fahrenheit. Demande à l'utilisateur de saisir la température en Celsius et affiche le résultat en Fahrenheit.

Exercice 7

Demande à l'utilisateur de saisir son âge et affiche un message indiquant s'il est majeur (18 ans ou plus) ou mineur.

Exercice 8

Demande à l'utilisateur de saisir deux nombres entiers et affiche le plus grand des deux.

Exercice 9

Demande à l'utilisateur de saisir une note (entre 0 et 100) et affiche la mention correspondante :

90-100 : "Excellent"

75-89 : "Bien"

50-74 : "Suffisant"

0-49 : "Insuffisant"

Exercice 10

Demande à l'utilisateur de saisir un nombre entre 1 et 7. Utilise une instruction switch pour afficher le jour de la semaine correspondant (1 = Lundi, 2 = Mardi, etc.).

Exercice 11

Demande à l'utilisateur de saisir un mot de passe et vérifie s'il correspond à un mot de passe prédéfini. Affiche un message indiquant si l'accès est accordé ou refusé.

Exercice 12

Demande à l'utilisateur de saisir les longueurs des trois côtés d'un triangle et vérifie si ces longueurs peuvent former un triangle valide.

Exercice 13

Demande à l'utilisateur de saisir le montant d'un achat. Si le montant est supérieur à 100 euros, applique une remise de 10 % et affiche le montant final.

Exercice 14

Crée une calculatrice simple qui demande à l'utilisateur deux nombres et une opération (addition, soustraction, multiplication ou division). Affiche le résultat selon l'opération choisie.

Exercice 15

Crée un tableau d'entiers contenant cinq valeurs. Affiche les valeurs du tableau sur la console.

Exercice 16

Demande à l'utilisateur de saisir 5 nombres entiers, stocke-les dans un tableau, puis calcule et affiche la somme de ces nombres.

Exercice 17

Crée un tableau d'entiers et demande à l'utilisateur de saisir un nombre. Vérifie si ce nombre se trouve dans le tableau et affiche un message approprié.

Exercice 18

Demande à l'utilisateur de saisir 5 nombres entiers, stocke-les dans un tableau, puis trie le tableau par ordre croissant et affiche les valeurs triées.

Exercice 19

Crée un tableau d'entiers et inverse les éléments du tableau. Affiche le tableau original et le tableau inversé.

Exercice 20

Demande à l'utilisateur de saisir 5 notes, stocke-les dans un tableau, puis calcule et affiche la moyenne des notes.

Exercice 21

Crée un tableau de 10 entiers. Remplis-le avec des valeurs aléatoires entre 1 et 10, puis compte et affiche combien de fois chaque nombre apparaît.

Exercice 22

Crée un système qui demande à l'utilisateur d'entrer plusieurs notes (jusqu'à ce qu'il saisisse -1 pour terminer).

Calcule la moyenne et affiche un message selon la moyenne :

90-100 : "Excellent"

75-89 : "Bien"

50-74 : "Suffisant"

0-49 : "Insuffisant"

Exercice 23

Crée un programme qui simule la gestion d'un compte bancaire. L'utilisateur peut choisir de déposer, retirer, ou afficher le solde. Inclure des vérifications pour s'assurer que les retraits ne dépassent pas le solde disponible.

Exercice 24

Demande à l'utilisateur de saisir un nombre et un intervalle (par exemple, de 1 à 10). Affiche la table de multiplication pour le nombre saisi, mais uniquement pour l'intervalle spécifié.

Exercice 25

Demande à l'utilisateur de saisir un nombre entier et vérifie s'il s'agit d'un nombre premier. Affiche un message en conséquence.

Exercice 26

Demande à l'utilisateur de saisir sa date de naissance au format JJ/MM. En fonction de la date, affiche son signe astrologique. (Saisi jour ; , saisir mois ; , traitement et affichage de l'horoscope

